Developer Productivity Measurement Platform

Metric design, architecture, and a 90-day plan

Context: ~8 squads, ~30 engineers, regulated fintech. This builds on the Part 1 prototype — a working GitHub-sourced dashboard computing seven metrics across two role-gated views, with its reasoning logged as eighteen numbered decisions (`D1` – `D18`), cited where relevant. Architecture diagram: final page.

---

1. Metric Design

Seven metrics ship today — a starting point, not a target (§1.2).

Metric	Shape	Source	Computation	Gaming defence
Deployment frequency	Count	Releases / deploy events	Published non-draft releases per week; report the **gap distribution** (p50/p85), never a mean	**Changes-per-deploy** alongside: splitting one shipment into three drives it toward 1 as frequency rises
Lead time for changes	Duration	Commits + deploy events	First commit (author date) → production; median and p85	Anchored on **first commit, not PR-open** (D17) — the ticket-branch convention makes the developer's own incentive start the clock honestly
Change failure rate	Ratio	Deploy events, issues, reverts	Failed ÷ total deploys, rolling 4 weeks. Failure = revert, incident-labelled issue in window, or hotfix shipped next (D5)	**Label coverage at equal visual weight** (D9): an unlabelled quarter reads *"0% — coverage 0%"*, never a clean green zero
PR size	Distribution	PR files	Lines changed after path exclusions; p50/p75/p90 and share over 400 lines	Paired with round trips — a size target produces stacked PRs, and the partner catches the wait moving rather than shrinking
Review round trips	Distribution	PR reviews + commits	Review-then-revise cycles per PR, humans only, normalised per 100 lines	**Both directions are suspicious**: high = unclear requirements; zero-and-falling while failure rate rises = rubber-stamping
Time to signal	Duration	CI runs + jobs	Push → first actionable verdict, split *time to red* / *time to green*, queue time broken out	`required_check_count` on the same chart, so a win from deleting tests is visible rather than celebrated
CI reliability	Ratio	CI run attempts	Rerun rate on unchanged SHAs; first-attempt pass rate; confirmed flake rate	Rerun rate is behavioural, first-attempt pass rate systemic: a team that gave up on CI scores well on one, badly on the other

Every defence is one of three kinds — **a tension partner** that degrades when the metric is gamed, **a companion figure** exposing the mechanism, or **never setting a target**.

The set is designed to grow. This dasboard and architecture is set to be living organism that grows with the organization on mulitple axes for example:

- **systems accessed** (work tracker → queue time by stage and investment profile; incident tooling → recovery time),
- **conventions imposed** (the ticket-branch rule; incident and hotfix labels), and

- **compliance signals** — specific to regulated fintech — such as unreviewed production change and segregation-of-duties breaches.

Deliberately excluded. Anything pointing at an **individual**, in any view, for any reason — no code path computes a per-person statistic (§2.4). Measurement is focused on **service** level and in other cases at **function** level and aggregated for **org** level.

---

2. Platform Architecture

See diagram, final page. Three stages: **ingestion → transformation → reporting.**

Ingestion holds every credential, API quirk and rate limit, and knows nothing about metrics. Two connector shapes share one contract — **push** (`subscribe` / `unsubscribe` / `verify` / `map` / `next`) and **pull** (`schedule` / `pull(cursor)` / `map` / `next`) — both terminating in `next()` , the seam where real-time alerting and batch transformation can be triggered. This boundary already exists in the prototype (`src/github-wrapper.js`): *GitHub is one source connector, not the product boundary.* **Transformation** reads an append-only event log, selecting on event type and labels, never payload content — scanning payloads to decide which matter is unaffordable, which makes labelling the ingest mapper's first job. Each transformer holds its own cursor, so all are independently resumable and replayable. **Reporting** serves two views from a signal store.

2.1 What makes it domain-neutral

Two abstractions, neither containing engineering vocabulary. **The event model is four nouns** — `entity` (what flows), `event` (a timestamped transition), `actor` (pseudonymised at ingestion if needed), `unit` (the owning group/function) — plus `window` , derived: a bounded interval between two events of the same kind, which is what every duration metric actually measures. And **every metric reduces to one of four shapes**: Count, Duration, Ratio, Distribution, with two variants, segmented duration and multi-class ratio. Each shape has one recipe the engine implements once, carrying its statistical commitments — medians not means, gap distributions not intervals, rolling not cumulative.

A new domain supplies events and definitions, coding is now more accessible than ever, any user can cotribute with connectors/ingestors/transformers/signals/etc.. through writing code not some complex markup.

Example for Part 1. **Review round trips is not an engineering metric**: it is a Distribution over review-then-revise cycles that happens to have been discovered in a code review, and any domain with a review loop already has it.

But additionally it extends to **Account opening time** is a Duration between `application.submitted` and `account.opened` on a customer id — same transformer, different event types. Or, a coffee machine emitting `empty` and `refilled` gets a signal, a dashboard and a permission model without touching the engine.

Most of this is adopted from battle tested best practices, which is the point. The event envelope is CloudEvents, the delivery vocabulary CDEvents, the pull contract Airbyte's; declarative definitions with row-level access control exist in Cube and dbt. Ours is the **Signal** — one object binding a metric to its shape, visualisation, permissions and provenance — plus push-ingestion lifecycle and governance-as-schema.

2.2 Who sees what, and why

Manager view (squad leads and engineering managers): distributions at p50/p75/p90 so shape is visible, and evidence down to the named PR, run or release — they decide what to unblock this week, which needs the specific artifact. **Executive view**: headline, band, direction and caveats, no evidence rows — investing next quarter needs trend and confidence. Direction uses two axes rather than a traffic light, because *healthy but degrading* is the early warning a single colour hides. Both resolve the same objects at three depths — **Signal → Number → Evidence** — under one rule: **the number is never hidden, only deferred.** An executive who wants the figure gets it with its caveats attached, rather than from a screenshot in a meeting without them.

2.3 Access, and the problem this org actually has

30 engineers across 8 squads is ~3.75 per squad. At that size a squad metric *is* an individual metric with extra steps, and "we never show individual grain" is no defence when everyone knows who wrote the big PRs. **So comparison is across services, never squads.** *"Checkout has a 90th-percentile lead time of nine days"* describes a system; the squad-

shaped version describes four named people. A squad always sees itself through owned services— self-view and comparative-view are different rights, not filters.

Four mechanisms enforce this, one per stage. **Pseudonymisation at ingestion** — identity becomes a salted hash inside the connector, and the identity map lives where the metric path cannot join to it, so a database compromise yields no per-person table because one was never assembled. **Individual grain is not computed** — absent, not hidden; hidden is one toggle from a leaderboard, and every engineer correctly assumes it exists. **Evidence is scoped and audited** — own-squad by default, cross-squad by explicit grant, every access logged, since a PR list is attributable however clean the aggregates are. And **the executive view doesn't include evidence by default**, structurally rather than by permission: a permission can be granted in a hurry before a board meeting; a missing code path cannot this avoids biased viewpoints.

Authentication is **organisational SSO via OIDC** (D18): IdP group claims supply both the audience model and squad membership, so access rules need no separate roster to drift. The platform holds no customer data, which changes which review it needs — and change traceability is evidence an auditor asks for regardless.

---

3. 90-Day Execution Plan

Solo builder. Each block ends with something real in someone's hands.

Days 1–5 — build-vs-buy spike, before writing anything

A single search pass found that most of this architecture is standardised, and that at least one open-source platform — **Faros Community Edition** — covers much of it: canonical model, Airbyte ingestion, push events API, 70+ connectors. **That search was quick, not decisive**, and planning 90 days of building without first spending a week trying to avoid it would be dishonest.

The spike tests Faros CE and the semantic-layer options against three questions: can it express the Signal object, can it enforce grain floors and drill-down permissions, and can one person operate it. **The plan then branches** — adopt, and the remaining days shift to configuration plus the novel third; build, and the sequence below runs as written. Either branch keeps the same day-90 number.

Days 6–30 — one squad, end to end

Ingestion → transformation → Manager view for a single squad including all services, four cheapest metrics. Ship to **one** squad and sit with them. The goal is not coverage; it is finding out whether the numbers survive contact with the people they describe. That is the only way data-quality problems surface, and they must surface while still cheap.

Days 31–60 — a platform rather than a script

Persistence lands as **the event log**, not a response cache, removing the prototype's re-query-on-every-refresh ceiling. Ingestion splits into push and pull shapes; definitions become declarative; pseudonymisation, SSO and the access model land here — before scale, not after. Expand to ~4 squads, add CI metrics.

Then **a second and third source: deployment events and the work tracker.** The highest-leverage integration in the plan, because deployment events carry a *status* that releases (as used in Part 1) structurally cannot — repairing change failure rate, unblocking recovery time, and closing the untagged-redeploy blind spot at once. It also supplies the evidence to design the canonical model from more than one example, which is how you avoid an abstraction shaped like its first source.

Days 61–90 — leadership, and proof of generality

Executive view, bands, noise gates, confidence gating, all 8 squads.

Adding new non-technical domains - whether user related, employee related or other functions as needed

Deliberately not built in 90 days

Incident-tool integration · surveys beyond the single trust question · architecture metrics (fan-in, criticality, blast radius) · AI-assist and authorship attribution · forecasting · alerting · self-serve metric authoring · multi-tenancy · mobile. All can come later once the framework is set and proven functional

The day-90 number

Metric trust score: the share of engineers who agree the dashboard reflects how work actually happens on their squad.

One question, five-point scale, anonymous, to all 30 engineers and 8 leads in week 12. **The denominator is all 38 invited, not respondents** — non-response counts as not trusting, which is the anti-gaming detail. ≥70% answering 4 or 5 means it works; below 50% means pulling the executive view. Paired with an objective falsifier: open unresolved data-quality disputes logged in-app. High trust alongside high disputes means people are being polite rather than convinced.

Not the alternatives: **adoption** (a dashboard can be opened and disbelieved), **DORA improvement** (measures the organisation, not the system; too slow at 90 days; invites gaming), **data coverage** (measures the pipeline — a perfectly covered dashboard nobody believes is still a failure). Trust is the necessary condition: if it is low, no other number should be acted on.

---

4. What I Would Change at 10x

At ~300 engineers and ~80 squads, the scale is different and full fledged framework is needed — the prototype computes every metric live, inside a single synchronous request.

The flaw is not the missing database. It is that **all three stages collapse into one request**: a page load fetches from GitHub, transforms and renders in one process. Correct for a one-week prototype with two users; wrong at scale for four reasons that bite together. One slow source API breaks the dashboard. History cannot be recomputed when a definition changes, because raw events were never kept — the provenance gap D11 and D14 already document. ~1,200 calls per load caps usage at four refreshes an hour. And ingestion (bursty I/O), transformation (batchable CPU) and reporting (must be fast) cannot be tuned in one process.

Replacement: separate the stages with durable boundaries — scheduled and webhook ingestion writing an append-only event log; batch transformation writing a signal store versioned by definition; reporting reading only that store. The boundaries matter more than the stores: each stage gets its own failure domain, scaling profile and replay path.

The strongest form of this answer is that **the reversal is incremental and its first step pays for itself immediately.** Persisting the event log — not a response cache — is small, survives untouched into the target architecture, unblocks additional sources, and delivers replay. Everything else follows later, on evidence.

TICKER

BAND 1 INGESTION source-specific, pluggable

GitHub | CI / Actions | HRIS | Core banking | IoT sensor

PUSH INGESTOR

`subscribe() · unsubscribe() · verify()`
`map() · next()`

webhook-driven

PULL INGESTOR

`schedule · pull(cursor)`
`map() · next()`

cron / polling, resumable

ACCESS CONTROL 1 · PSEUDONYMISATION — actor identity is hashed here, before anything downstream

EVENT LOG

append-only, ever-increasing

`id · stream_seq · time · type · version · labels · subject · actor · payload`

CloudEvents-shaped. `labels` *and* `type` *are the only selector surface — payloads are never scanned.*

BAND 2 TRANSFORMATION domain-neutral. This is the product.

▲ **TRANSFORMER**
`selector { types, labels }`
`cursor → stream_seq`
`shape`

▲ **TRANSFORMER**
`selector { types, labels }`
`cursor → stream_seq`
`shape`

▲ **TRANSFORMER**
`selector { types, labels }`
`cursor → stream_seq`
`shape`

THE FOUR SHAPES

Count — how much completed

Duration — how long between two points

Ratio — what share satisfies a condition

Distribution — shape of a per-entity quantity

+2 variants: segmented duration, multi-class ratio

ACCESS CONTROL 2 — individual grain is not computed; the engine has no code path for it

provenance: selector + seq_range + exception ids

SIGNALS

a signal is a metric

`shape · value · grain · scope · window · confidence · band · direction · visualization · permission · drilldown_permission · provenance`

BAND 3 REPORTING domain-neutral shell

▲ WIDGET
accepts Distribution

▲ WIDGET
accepts Duration

▲ WIDGET
accepts n signals
(tension pair)

VIEW 1

Layer 1 Signal
Layer 2 Number
Layer 3 Evidence

VIEW 2

Layer 1 Signal
Layer 2 Number
~~Layer 3 Evidence~~

DEPENDS ON THE PERMISSIONS

ACCESS CONTROL 3 — evidence access is scoped to own unit and audit-logged

one clock drives everything recurring